Module 2 - Operators:

1. Explain arithmetic operator with example

Arithmetic operators in Java are used to perform basic mathematical operations like addition, subtraction, multiplication, and more. They're fundamental when working with numbers.

The minus operator also has a unary form that negates its single operand. Remember that when the division operator is applied to an integer type, there will be no fractional component attached to the result.

Operator	Description	Example Expression	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (remainder)	10 % 3	1

```
class BasicMath {
public static void main(String args[]) {
 System.out.println("Integer Arithmetic");
int a = 1 + 1;
int b = a * 3;
int c = b / 4;
int d = c - a;
int e = -d;
System.out.println("a = " + a);
System.out.println("b = " + b);
System.out.println("c = " + c);
System.out.println("d = " + d);
System.out.println("e = " + e);
System.out.println("\nFloating Point Arithmetic");
double da = 1 + 1;
double db = da * 3;
double dc = db / 4;
double dd = dc - a;
double de = -dd;
System.out.println("da = " + da);
System.out.println("db = " + db);
System.out.println("dc = " + dc);
System.out.println("dd = " + dd);
```

```
System.out.println("de = " + de);
}
}
```

Output

Integer Arithmetic a = 2 b = 6 c = 1 d = -1 e = 1
Floating Point Arithmetic da = 2.0 db = 6.0
dc = 1.5 dd = -0.5 de = 0.5

2. Explain Bitwise operators. Or Explain the following operators with example And(&) Or(|) Right shift(>>) Left Shift(<<)

Java defines several *bitwise operators* that can be applied to the integer types, **long**, **int**, **short**, **char**, and **byte**. These operators act upon the individual bits of their operands.

Operator	Symbol	Description	Example	Result
AND	&	Sets each bit to 1 if both bits are 1	5 & 3 → 101 & 011	001 (1)
OR	|	Sets each bit to 1 if at least one is 1	5 | 3 → 101 | 011	111 (7)
XOR	^	Sets each bit to 1 if bits are different	5 ^ 3 → 101 ^ 011	110 (6)
NOT	~	Inverts all bits	~5 → ~00000101	Typically -6 (in 2's complement)
Left Shift	<<	Shifts bits to the left	5 << 1 → 1010	10
Right Shift	>>	Shifts bits to the right, preserves sign	5 >> 1 → 10	2
Unsigned Right Shift	>>>	Shifts right, fills with zeros	-5 >>> 1	Large positive value

```
public class BitwiseDemo {
   public static void main(String[] args) {
      int a = 5;  // Binary: 0101
      int b = 3;  // Binary: 0011

      System.out.println("AND: " + (a & b));      // 1
      System.out.println("OR: " + (a | b));       // 7
      System.out.println("XOR: " + (a ^ b));      // 6
      System.out.println("NOT: " + (~a));         // -6
      System.out.println("Left Shift: " + (a << 1)); // 10
      System.out.println("Right Shift: " + (a >> 1)); // 2
   }
```

}

The Left Shift

The left shift operator, **<<**, shifts all of the bits in a value to the left a specified number of times. It has this general form:

value << *num*

Here, *num* specifies the number of positions to left-shift the value in *value.* That is, the **<<** moves all of the bits in the specified value to the left by the number of bit positions specified by *num.* For each shift left, the high-order bit is shifted out (and lost), and a zero is brought in on the right. This means that when a left shift is applied to an **int** operand, bits are lost once they are shifted past bit position 31. If the operand is a **long**, then bits are lost after bit position 63.

```
class ByteShift { public static void main(String args[]) {
byte a = 64, b; int i;
i = a << 2; b = (byte) (a << 2);
System.out.println("Original value of a: " + a);
System.out.println("i and b: " + i + " " + b);
}
}
```

The output generated by this program is shown here:

Original value of a: 64 i and b: 256 0

The Right Shift

The right shift operator, **>>**, shifts all of the bits in a value to the right a specified number of times. Its general form is shown here:

value >> *num*

Here, *num* specifies the number of positions to right-shift the value in *value.* That is, the **>>** moves all of the bits in the specified value to the right the number of bit positions specified by *num.*

The following code fragment shifts the value 32 to the right by two positions, resulting in **a** being set to 8:

```
int a = 35;
a = a >> 2; // a still contains 8
```

Looking at the same operation in binary shows more clearly how this happens:

```
00100011        35
>> 2
00001000         8
```

3. List and explain relational operators.

The *relational operators* determine the relationship that one operand has to the other. Specifically, they determine equality and ordering. The relational operators are shown here:

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than

<	Less than
>=	Greater than or equal to
<=	Less than or equal to

The outcome of these operations is a **boolean** value. The relational operators are most frequently used in the expressions that control the **if** statement and the various loop statements.

```
public class RelationalDemo {
   public static void main(String[] args) {
      int x = 10, y = 20;

      System.out.println("x == y : " + (x == y));
      System.out.println("x != y : " + (x != y));
      System.out.println("x > y  : " + (x > y));
      System.out.println("x < y  : " + (x < y));
      System.out.println("x >= y : " + (x >= y));
      System.out.println("x <= y : " + (x <= y));
   }
}
```

4. Explain Boolean logical operators with an example

The Boolean logical operators shown here operate only on **boolean** operands. All of the binary logical operators combine two **boolean** values to form a resultant **boolean** value.

Operator	Result
&	Logical AND
|	Logical OR
^	Logical XOR (exclusive OR)
||	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
|=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

The logical Boolean operators, **&**, |, and ^, operate on **boolean** values in the same way that they operate on the bits of an integer. The logical **!** operator inverts the Boolean state:
!true == false and **!false == true**.
The following table shows the effect of each logical operation:

A	B	A | B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

Here is a program that is almost the same as the **BitLogic** example shown earlier, but it operates on **boolean** logical values instead of binary bits:

```
// Demonstrate the boolean logical operators. class BoolLogic {
public static void main(String args[]) {
boolean a = true; boolean b = false; boolean c = a | b; boolean d = a & b; boolean e = a ^ b; boolean f = (!a & b) | (a & !b); boolean g = !a; System.out.println("        a = " + a);
System.out.println("        b = " + b);
System.out.println("      a|b = " + c);
System.out.println("      a&b = " + d);
System.out.println("      a^b = " + e);
System.out.println("!a&b|a&!b = " + f);
System.out.println("       !a = " + g);
}
}
```

After running this program, you will see that the same logical rules apply to **boolean** values as they did to bits. As you can see from the following output, the string representation of a Java **boolean** value is one of the literal values **true** or **false**:

```
a = true b = false a|b = true a&b = false a^b = true a&b|a&!b = true !a = false
```

5. Explain Assignment Operator.

The *assignment operator* is the single equal sign, **=**. The assignment operator works in Java much as it does in any other computer language. It has this general form:

var = expression;

Here, the type of *var* must be compatible with the type of *expression.*

The assignment operator does have one interesting attribute that you may not be familiar with: it allows you to create a chain of assignments. For example, consider this fragment:

```
int x, y, z; x = y = z = 100; // set x, y, and z to 100
```

This fragment sets the variables **x**, **y**, and **z** to 100 using a single statement. This works because the **=** is an operator that yields the value of the right-hand expression. Thus, the value of **z = 100** is 100, which is then assigned to **y**, which in turn is assigned to **x**. Using a "chain of assignment" is an easy way to set a group of variables to a common value.

6. Explain ? Operator

Java includes a special *ternary* (three-way) *operator* that can replace certain types of if-then-else statements. This operator is the **?**. It can seem somewhat confusing at first, but the **?** can be used very effectively once mastered. The **?** has this general form:

expression1 **?** *expression2* **:** *expression3*

Here, *expression1* can be any expression that evaluates to a **boolean** value. If *expression1* is **true**, then *expression2* is evaluated; otherwise, *expression3* is evaluated. The result of the **?** operation is that of the expression evaluated. Both *expression2* and *expression3* are required to return the same type, which can't be **void**.

Here is an example of the way that the **?** is employed:

```
ratio = denom == 0 ? 0 : num / denom;
```

When Java evaluates this assignment expression, it first looks at the expression to the *left* of the question mark. If **denom** equals zero, then the expression *between* the question mark and the colon is evaluated and used as the value of the entire **?** expression. If **denom** does not equal zero, then the expression *after* the colon is evaluated and used for the value of the entire **?** expression. The result produced by the **?** operator is then assigned to **ratio**.

Here is a program that demonstrates the **?** operator. It uses it to obtain the absolute value of a variable.

```
class Ternary {
public static void main(String args[]) {
int i, k;
i = 10;
k = i < 0 ? -i : i;
System.out.print("Absolute value of ");
System.out.println(i + " is " + k);
i = -10;
k = i < 0 ? -i : i;
System.out.print("Absolute value of ");
System.out.println(i + " is " + k);
}
}
```

The output generated by the program is shown here:

```
Absolute value of 10 is 10
Absolute value of -10 is 10
```

7. Explain four different types of if statements in JAVA.

1. Simple if Statement

Executes a block of code only if a condition is true.

```
int age = 18;
if (age >= 18) {
System.out.println("You are eligible to vote."); }
```

✔ **Output**: You are eligible to vote.

2. if-else Statement

Runs one block if the condition is true, another block if false.

```
int number = -5;
if (number > 0) {
System.out.println("Positive number.");
}
```

```
else {
 System.out.println("Non-positive number.");
}
```

✔ **Output**: Non-positive number

3. if-else if-else Ladder

Chains multiple conditions. Only the first true condition's block runs.

```
int marks = 85;
if (marks >= 90) {
System.out.println("Grade: A");
}
else if (marks >= 75) {
System.out.println("Grade: B");
}
else if (marks >= 60) {
System.out.println("Grade: C");
}
else {
System.out.println("Grade: F");
}
```

✔ **Output**: Grade:

4. Nested if Statements

An if inside another if. Used for more complex logic checks.

```
int age = 25;
 boolean hasID = true;
if (age >= 18) {
if (hasID) {
System.out.println("Access granted.");
}
else {
System.out.println("ID required for access.");
}
}
else {
System.out.println("Access denied. Age restriction.");
}
```

✔ **Output**: Access granted.

8. Explain Switch Statement with an example.

The **switch** statement is Java's multiway branch statement. It provides an easy way to dispatch execution to different parts of your code based on the value of an expression. As such, it often provides a better alternative than a large series of **if-else-if** statements. Here is the general form of a **switch** statement:

```
switch (expression) {
case value1:
// statement sequence break;
case value2:
// statement sequence break;
.
.
. case valueN: // statement sequence break;
default:
// default statement sequence
}
```

Example:

```
class SampleSwitch {
public static void main(String args[]) {
for(int i=0; i<6; i++) switch(i) {
case 0:
System.out.println("i is zero."); break; case 1:
System.out.println("i is one."); break; case 2:
System.out.println("i is two."); break; case 3:
System.out.println("i is three."); break; default:
System.out.println("i is greater than 3.");
}
}
}
```

The output produced by this program is shown here:

```
i is zero.
i is one.
i is two.
i is three.
i is greater than 3.
i is greater than 3.
```

9. Explain Iteration Statements.

Java's iteration statements are **for**, **while**, and **do-while**. These statements create what we commonly call *loops.* As you probably know, a loop repeatedly executes the same set of instructions until a termination condition is met.

while

The **while** loop is Java's most fundamental loop statement. It repeats a statement or block while its controlling expression is true. Here is its general form:

```
while(condition) {
// body of loop
}
```

The *condition* can be any Boolean expression. The body of the loop will be executed as long as the conditional expression is true. When *condition* becomes false, control passes to the next line of code immediately following the loop. The curly braces are unnecessary if only a single statement is being repeated.

Here is a **while** loop that counts down from 10, printing exactly ten lines of "tick":

```
// Demonstrate the while loop. class While { public static void main(String args[]) {
int n = 10;
while(n > 0) {
System.out.println("tick " + n); n--;
}
}
}
```

Output:

```
tick 10 tick 9 tick 8 tick 7 tick 6 tick 5 tick 4 tick 3 tick 2 tick 1
```

do-while

As you just saw, if the conditional expression controlling a **while** loop is initially false, then the body of the loop will not be executed at all. However, sometimes it is desirable to execute the body of a loop at least once, even if the conditional expression is false to begin with. In other words, there are times when you would like to test the termination expression at the end of the loop rather than at the beginning. Fortunately, Java supplies a loop that does just that: the **do-while**. The **do-while** loop always executes its body at least once, because its conditional expression is at the bottom of the loop. Its general form is

```
do {
// body of loop
} while (condition);
```

Each iteration of the **do-while** loop first executes the body of the loop and then evaluates the conditional expression. If this expression is true, the loop will repeat. Otherwise, the loop terminates. As with all of Java's loops, *condition* must be a Boolean expression.

Here is a reworked version of the "tick" program that demonstrates the **do-while** loop.

It generates the same output as before.

```
// Demonstrate the do-while loop. class DoWhile { public static void main(String args[]) {
int n = 10;
do {
System.out.println("tick " + n); n--;
} while(n > 0);
}
}
```

for

You were introduced to a simple form of the **for** loop in Chapter 2. As you will see, it is a powerful and versatile construct.

Beginning with JDK 5, there are two forms of the **for** loop. The first is the traditional form that has been in use since the original version of Java. The second is the new "for-each" form.

Both types of **for** loops are discussed here, beginning with the traditional form. Here is the general form of the traditional **for** statement:

for(*initialization*; *condition*; *iteration*) {
// body
}

If only one statement is being repeated, there is no need for the curly braces.

The **for** loop operates as follows. When the loop first starts, the *initialization* portion of the loop is executed. Generally, this is an expression that sets the value of the *loop control variable*, which acts as a counter that controls the loop. It is important to understand that the initialization expression is only executed once. Next, *condition* is evaluated. This must be a Boolean expression. It usually tests the loop control variable against a target value. If this expression is true, then the body of the loop is executed. If it is false, the loop terminates. Next, the *iteration* portion of the loop is executed. This is usually an expression that increments or decrements the loop control variable. The loop then iterates, first evaluating the conditional expression, then executing the body of the loop, and then executing the iteration expression with each pass. This process repeats until the controlling expression is false.

Here is a version of the "tick" program that uses a **for** loop:

```
// Demonstrate the for loop.
class ForTick {
public static void main(String args[]) {
int n;
for(n=10; n>0; n--)
System.out.println("tick " + n);
}
}
```

10. Explain Jump Statements

Java supports three jump statements: **break**, **continue**, and **return**. These statements transfer control to another part of your program.

Using break

In Java, the **break** statement has three uses. First, as you have seen, it terminates a statement sequence in a **switch** statement. Second, it can be used to exit a loop. Third, it can be used as a "civilized" form of goto. The last two uses are explained here.

Using break to Exit a Loop

By using **break**, you can force immediate termination of a loop, bypassing the conditional expression and any remaining code in the body of the loop. When a **break** statement is encountered inside a loop, the loop is terminated and program control resumes at the next statement following the loop. Here is a simple example:

```
// Using break to exit a loop.
class BreakLoop {
public static void main(String args[]) {
for(int i=0; i<100; i++) {
if(i == 10) break; // terminate loop if i is 10 System.out.println("i: " + i);
}
```

```
System.out.println("Loop complete.");
}
}
```
This program generates the following output:
```
i: 0 i: 1 i: 2 i: 3 i: 4 i: 5 i: 6 i: 7 i: 8 i: 9 Loop complete.
```
Using continue

Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration. This is, in effect, a goto just past the body of the loop, to the loop's end. The **continue** statement performs such an action. In **while** and **do-while** loops, a **continue** statement causes control to be transferred directly to the conditional expression that controls the loop. In a **for** loop, control goes first to the iteration portion of the **for** statement and then to the conditional expression. For all three loops, any intermediate code is bypassed.

Here is an example program that uses **continue** to cause two numbers to be printed on each line:
```
// Demonstrate continue.
class Continue {
public static void main(String args[]) {
for(int i=0; i<10; i++) {
System.out.print(i + " "); if (i%2 == 0) continue; System.out.println("");
}
}
}
```
This code uses the **%** operator to check if **i** is even. If it is, the loop continues without printing a newline. Here is the output from this program:
```
0 1
2 3
4 5
6 7
8 9
```
return

The last control statement is **return**. The **return** statement is used to explicitly return from a method. That is, it causes program control to transfer back to the caller of the method. As such, it is categorized as a jump statement. Although a full discussion of **return** must wait until methods are discussed in Chapter 6, a brief look at **return** is presented here.

At any time in a method the **return** statement can be used to cause execution to branch back to the caller of the method. Thus, the **return** statement immediately terminates the method in which it is executed. The following example illustrates this point. Here, **return** causes execution to return to the Java run-time system, since it is the run-time system that calls **main( )**.
```
// Demonstrate return.
class Return {
public static void main(String args[]) {
boolean t = true;
System.out.println("Before the return."); if(t) return; // return to caller
```

```
System.out.println("This won't execute.");
}
}
```
The output from this program is shown here:
Before the return.

11. Write a program to sort the elements using for loop.

```
public class ArraySortExample {
public static void main(String[] args) {
int[] arr = {5, 2, 8, 1, 3};
int temp;
for (int i = 0; i < arr.length - 1; i++) {
for (int j = 0; j < arr.length - i - 1; j++) {
if (arr[j] > arr[j + 1]) {
// Swap elements
temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp;
}
}
}

System.out.println("Sorted array in ascending order:");
for (int num : arr) {
System.out.print(num + " ");
}
}
}
```

Output:
Sorted array in ascending order:
1 2 3 5 8

12. Write a program to find the biggest of three numbers using ternary operator.

```
public class BiggestOfThree {
public static void main(String[] args) {
int a = 42;
int b = 67;
int c = 13;

int largest = (a > b) ?
((a > c) ? a : c) :
((b > c) ? b : c);
```

```
System.out.println("The biggest number is: " + largest);
}
}
```

Output

```
The biggest number is: 67
```